T2

Multilayer Perceptron

Salvador Ruiz Correa

March 24, 2026

A MULTILAYER PERCEPTRON (MLP) is a type of artificial neural network (ANN) consisting of multiple layers of nodes, with each layer fully connected to the next layer. It typically comprises an input layer, one or more hidden layers, and an output layer. Each node in the hidden layers and the output layer applies a nonlinear activation function to the weighted sum of its inputs, allowing the network to learn complex relationships in the data. MLPs are commonly used for supervised learning tasks such as classification and regression. They are trained using techniques like backpropagation, adjusting the weights of connections between nodes to minimize the difference between the predicted output and the actual output.

AGENDA:

1 Multilayer Perceptron.
2 Backpropagation.
3 Gradient Descent.
4 Curse of Dimensionality.
5 Separation Theorem.
5 Kernel Trick

Regression Model

We start by discussing regression problems. We begin by considering a generative model where the target forma a multivariate Gaussian vector $\mathcal{N}(\mathbf{t} \mid \boldsymbol{\mu}(\mathbf{x}), \beta^{-1}\mathbf{I})$, where $\mathbf{I}$ is a diagonal matrix and β is the precision (inverse of the variance). We are given a set of input vectors $\{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_N\}$ and the corresponding target vectors $\{\mathbf{T}_1, \mathbf{T}_2, \ldots, \mathbf{T}_N\}$. See Figure for a simple example.

To solve the regression model, we approximate the mean vector $\mu(\mathbf{x})$ with the random vectors $\mathbf{Y}$ as depicted given by the following probability density for the target variables is $\mathcal{N}(\mathbf{t} \mid \boldsymbol{y}(\mathbf{x}, \mathbf{w}), \beta^{-1}\mathbf{I})$.

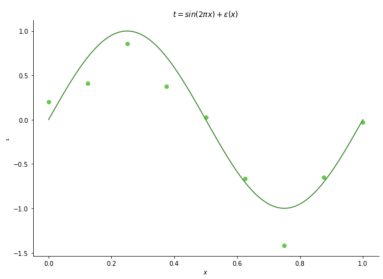
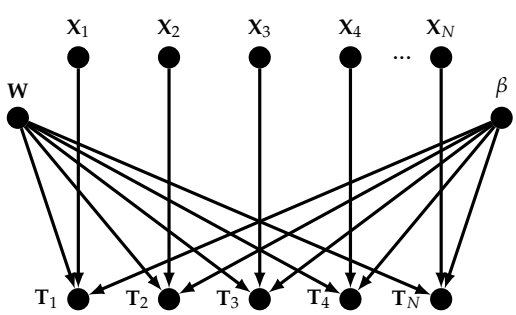


Figure 1: Green dots represent data samples computed as $t = \sin(2\pi x) + \epsilon(x)$, for a set of x input data where ϵ represents a normal *random noise* of mean zero and variance σ^2. Notice that the probability density of the target variable is $\mathcal{N}(t \mid \sin(2\pi x), \sigma^2)$.



We will use a neural network model that uses a solution space of the form:

$$\mathcal{H}_{\mathbf{w}} = \{y_k(\mathbf{x}_n, \mathbf{w}) \ , k = 1, 2, \ldots, K\},$$

where

$$y_k(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{M_L} w_{kj}^L \, h^L \left(\cdots \sum_{r=0}^{M_2} w_{sr}^2 \, h^2 \left(\sum_{q=0}^{M_1} w_{rq}^1 \, h^1 \left(\sum_{i=0}^{M_0} w_{qi}^0 x_i \right) \right) \right).$$

and h^l, $l = 1, 2, \ldots, L$, is an activation function. Notice that the term

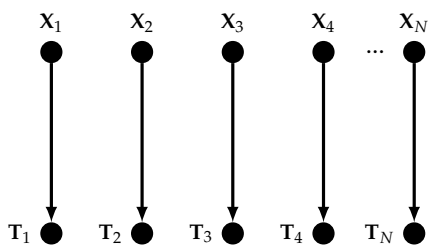
$$h^L \left(\cdots \sum_{r=0}^{M_2} w_{sr}^2 \, h \left(\sum_{q=0}^{M_1} w_{rq}^1 \, h \left(\sum_{i=0}^{M_0} w_{qi}^0 x_i \right) \right) \right)$$

depends on the j index. Notice that there is a total of

$$\sum_{l=0}^{L+1} M_l \times M_{l-1}$$

random variables $\mathbf{W}$.

First, we will use the frequentist approach to solve the regression problem using a neural network by assuming that the vectors $\mathbf{w}$ and β are parameters that need to be estimated using the MLE approach.



That is, we have that the target vector $\mathbf{t}$ is distributed according to $\mathcal{N}(\mathbf{t}; \boldsymbol{y}(\mathbf{x}, \mathbf{w}), \beta^{-1}\mathbf{I})$, where $\mathbf{w}$ and β are the probability density parameters.

[o] Source:Bishop. (2006). Pattern Recognition and Machine Learning, Springer.

Multilayer Perceptron

We introduce the MLP function family using a simple example. Assume that we have an i.i.d. data set $\mathcal{D} = \{(\mathbf{x}_1, \mathbf{t}_1), (\mathbf{x}_2, \mathbf{t}_2), \ldots, (\mathbf{x}_N, \mathbf{t}_N)\}$ where $\mathbf{x}_n = (x_{n1}, x_{n2})$, and $\mathbf{t}_n = (t_{n1}, t_{n2})$. The probability density of $\mathbf{T}_n$ given $\mathbf{X}_n$ is $p(\mathbf{t}_n \mid \mathbf{x}_n) = \mathcal{N}(\mathbf{t}_n; \mathbf{y}(\mathbf{x}_n, \mathbf{w}), \sigma^2\mathbf{I}))$ where $\beta = \sigma^{-2}$ is the precision.

The likelihood of the data is

$$\prod_{n=1}^{N} p(\mathbf{t}_n \mid \mathbf{x}_n) = \prod_{n=1}^{N} (2\pi)^{-1} \beta \exp\left(-\frac{\beta}{2} \mid\mid \mathbf{t}_n - \mathbf{y}(\mathbf{x}_n, \mathbf{w}) \mid\mid^2 \right).$$

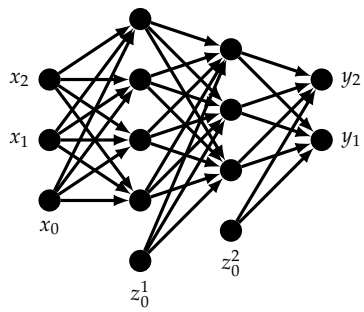


Figure 2: Multilayer perceptron used to represent the function $\mathbf{y}(\cdot,\cdot)$.

The negative of the likelihood is

$$\begin{aligned}
\mathrm{E}(\mathbf{w},\beta) &= -\log \prod_{n=1}^{N} p(\mathbf{t}_n \mid \mathbf{x}_n) \\
&= \frac{\beta}{2} \sum_{n=1}^{N} \mid\mid \mathbf{t}_n - \mathbf{y}(\mathbf{x}_n,\mathbf{w}) \mid\mid^2 - N \log \beta + N \log(2\pi) \\
&= \frac{\beta}{2} \sum_{n=1}^{N} \sum_{i=1}^{2} (t_{ni} - y_i(\mathbf{x}_n,\mathbf{w}))^2 - N \log \beta + N \log(2\pi)
\end{aligned}$$

The mean vector of the distribution is represented by the function $\mathbf{y}(\cdot,\cdot)$, which corresponds to the computational structure depicted in Figure 2.

This neural network has $L = 4$ layers $(0,1,2,3)$, including the input, output, and two hidden layers. For the reasons below, we use the auxiliary variables x_0, z_0^1, and $z2_0$. The i-th node of one layer (i.e., the i-th artificial neuron) connects to the j row of the consecutive layer (say layer l) through a weight

$$w_{ji}^l = w_{i\to j}^l.$$

Nodes x_0, z_0^1, and z_0^2 do not receive connections from the previous layer, as shown in Figure 2. Notice that each neuron in the hidden and output layers is fully connected to the neurons of the previous layer.

The key idea is to find the network weights that minimize the data's minus log-likelihood. Namely, we want to find an MLE solution to the problem.

For now, we assume that the precision β is given. Therefore, we want to minimize:

$$\begin{aligned}
\mathrm{E}(\mathbf{w}) &= \frac{1}{2} \sum_{n=1}^{N} \sum_{i=1}^{2} (t_{ni} - y_i(\mathbf{x}_n,\mathbf{w}))^2 \\
&= \sum_{n=1}^{N} \mathrm{E}_n(\mathbf{w}).
\end{aligned}$$

To find the MLE solution using SGD, we need to understand how data is processed in the network and how to compute the gradient of $\mathrm{E}(\mathbf{w})$. In what follows, we omit the subscript n to avoid cluttering notation.

Forward Propagation

We start by computing the output of each neuron in the hidden and the output layers, often called forward propagation or forward pass. Each output (say the j-the neuron in of the l layer) is calculated in two stages.

- First, incoming data from the previous layer is transformed using an affine transformation.

$$\begin{aligned} a_j^l &= \sum_{\forall i \to j} w_{ji}^l z_i^{l-1} \\ &= \sum_{i=0}^{M_l - 1} w_{ji}^l z_i^{l-1} \\ &= \sum_{i=1}^{M_l - 1} w_{ji}^l z_i^{l-1} + z_0^{l-1} w_{j0}^l \\ &= \sum_{i=1}^{M_l - 1} w_{ji}^l z_i^{l-1} + w_{j0}^l \\ &= \sum_{i=1}^{M_l - 1} w_{ji}^l z_i^{l-1} + b_j^l, \end{aligned} \tag{1}$$

  with $l = 1, 2, 3$, and where $z_i^0 = x_i$, and $z_0^{l-1} = 1$. The b_j^l variable is often called bias. In matrix notation,

$$\mathbf{A}^l = \mathbf{W}^l \mathbf{Z}^{l-1}.$$

  $\mathbf{A}^l$ is a column vector. Note that

$$\mathbf{A}^l = \mathbf{W}^l \mathbf{W}^{l-1} \cdots \mathbf{W}^1 \mathbf{x},$$

  corresponds to the forward propagation of $\mathbf{x}$ to the l-th layer for a linear MLP. It can be as deep as one wants.
- Second, the output of the affine transformation is input to a non-linear, differentiable function (i.e., the *activation function*, $h^l(\cdot)$) to compute the output

$$z_j^l = h^l(a_j^l).$$

- In the regression case, the activation function is the identity function, so the output of each neuron in the output layer is

$$y_i = z_i^L = a_i^L$$

  .

Backward Propagation

Backward Propagation or backpropagation aims to find a recursion equation that enables computation of the gradient of $\mathrm{E}(\mathbf{w})$,

$$\nabla \mathrm{E}(\mathbf{w}) = \sum_{n=1}^{N} \nabla \mathrm{E}_n(\mathbf{w}).$$

We require the *chain rule for partial derivatives* to derive the recursion equation.

Box 1: Chain Rule for Partial Derivatives

Suppose that variable u is a function of N variables $v_1, v_2, \ldots, v_N$, and that each of these is, in turn, a function of m variables $w_1, w_2, \ldots, w_M$. Then for any variable w_i, $i = 1, 2, \ldots, M$ we have the following

$$\frac{\partial u}{\partial w_i} = \sum_{n=1}^{N} \frac{\partial u}{\partial v_n} \frac{\partial v_n}{\partial w_i}$$

As an example, lets compute $\frac{\partial \mathrm{E}_n(\mathbf{w})}{\partial z_j^l}$. Recall that for the hidden and output layers (see see Eq. (1)),

$$a_k^{l+1} = \sum_{\forall i \to j} w_{ki}^{l+1} z_i^l.$$

In matrix notation

$$\mathbf{A}^{l+1} = \mathbf{W}^{l+1} \mathbf{Z}^l.$$

Also, notice that $\mathrm{E}_n(\mathbf{w})$ is a function of a_k^{l+1} for each of the neurons in the $(l+1)$-th layer. Therefore

$$\begin{aligned}
\frac{\partial E_n}{\partial z_j^l} &= \sum_{k=1}^{M_l} \frac{\partial E_n}{\partial a_k^{l+1}} \frac{\partial a_k^{l+1}}{\partial z_j^l} \\
&= \sum_{k=1}^{M_l} d_k^{l+1} \frac{\partial a_k^{l+1}}{\partial z_j^l} \\
&= \sum_{k=1}^{M_l} d_k^{l+1} \frac{\partial}{\partial z_j^l} \left[\sum_{i=1}^{M_l} w_{ki}^{l+1} z_i^l \right] \\
&= \sum_{k=1}^{M_l} d_k^{l+1} w_{kj}^{l+1}. \qquad (2)
\end{aligned}$$

In matrix notation

$$\partial \mathbf{Z}^l = (\mathbf{W}^{l+1})^T \mathbf{d}^{l+1}.$$

Now we will compute the gradient components $\frac{\partial E_n}{\partial w_{ji}^l}$. Again, we omit the subscript n from the network variables to keep the notation uncluttered.

$$\begin{aligned}\frac{\partial E_n}{\partial w_{ji}^l} &= \frac{\partial E_n}{\partial a_j^l}\frac{\partial a_j^l}{\partial w_{ji}^l} \\ &= d_j^l z_i^{l-1}.\end{aligned}$$

Notice that we defined d_j^l as

$$d_j^l = \frac{\partial E_n}{\partial a_j^l}.$$

The gradient component $\frac{\partial E_n}{\partial w_{ji}^l}$ can be computed using the following recursion equation:

$$\begin{aligned}d_j^l &= \frac{\partial E_n}{\partial a_j^l} \\ &= \frac{\partial E_n}{\partial z_j^l}\frac{\partial z_j^l}{\partial a_j^l} \\ &= \frac{\partial E_n}{\partial z_j^l} h'(a_j^l) \\ &= h^{l'}(a_j^l)\sum_{k=1}^{M_l} d_k^{l+1} w_{kj}^{l+1}.\end{aligned}$$

In matrix notation

$$\mathbf{d}^l = \partial\mathbf{A}^l \odot \partial\mathbf{Z}^l = \partial\mathbf{A}^l \odot (\mathbf{W}^{l+1})^T\mathbf{d}^{l+1} = \mathrm{diag}(\partial\mathbf{A}^l)(\mathbf{W}^{l+1})^T\mathbf{d}^{l+1}$$

$$\mathbf{d}^l = \partial\mathbf{A}^l \odot \partial\mathbf{Z}^l = \underbrace{\partial\mathbf{A}^l \odot (\mathbf{W}^{l+1})^T}_{\text{error propagator}}\mathbf{d}^{l+1}$$

where $\odot$ denotes the hadamard product, and the j-th component of $\partial\mathbf{A}^l$ is $h^{l'}(a_j^l)\sum_{k=1}^{M_l+1} d_k^{l+1} w_{kj}^{l+1}$. Recall that:

$$\frac{\partial E_n}{\partial z_j^l} = \sum_{k=1}^{M_l+1} d_k^{l+1} w_{kj}^{l+1},$$

and

$$\begin{aligned}\frac{\partial z_j^l}{\partial a_j^l} &= \frac{dh^l(a_j^l)}{da_j^l} \\ &= h^{l'}(a_j^l).\end{aligned}$$

Finally, the gradients are computed as

$$\partial \mathbf{W}^l = \mathbf{d}^l (\mathbf{Z}^{l-1})^T$$

Box 2: Backpropagation: Gradient Computation

1. Apply an input vector x_n to the network and forward propagate through the network to find the activations of all the hidden and output units.
2. Evaluate the d's for all the output units: $d_j^L = y_j - t_j$.
3. Backpropagate the d's to obtain the d'sfor each hidden unit in the network.

$$d_j^l \leftarrow h^{l\prime}(a_j^l) \sum_{k=1}^{M_l+1} d_k^{l+1} w_{kj}^{l+1}.$$

$$\begin{aligned} \mathbf{d}^l &= \partial \mathbf{A}^l \odot (\mathbf{W}^{l+1})^T \mathbf{d}^{l+1} \\ &= \operatorname{diag}(\partial \mathbf{A}^l)(\mathbf{W}^{l+1})^T \mathbf{d}^{l+1} \end{aligned}$$

4. Evaluate the required derivatives.

$$\frac{\partial E_n}{\partial w_{ji}^l} = d_j^l z_i^{l-1}.$$

$$\partial \mathbf{W}^l = \mathbf{d}^l (\mathbf{Z}^{l-1})^T$$

Why is $\frac{\partial E_n}{\partial z_j^L} = y_j - t_j$? Recall that for the given error function, we have that

$$\begin{aligned} \mathrm{E}_n(\mathbf{w}) &= \frac{1}{2} \sum_{j=1}^{2} \left(t_j - y_j(\mathbf{x}, \mathbf{w})\right)^2 \\ &= \frac{1}{2} \sum_{j=1}^{2} \left(y_j(\mathbf{x}, \mathbf{w}) - t_j\right)^2 \\ &= \frac{1}{2} \sum_{j=1}^{2} \left(z_j^L - t_j\right)^2. \end{aligned}$$

In the regression setting the activation function is the identity function, so $z_j^L = a_j^L$.

Example

Consider an MLP with two hidden layers such that $M_0 = 2$, $M_1 = 4$, $M_2 = 3$, and $M_3 = 2$. The backpropagation update equations are the

following. Notice that the di9mensions of the weight matrices $\mathbf{W}^3$, $\mathbf{W}^2$, $\mathbf{W}^1$ are $(2,4)$, $(4,5)$ and $(5,3)$ respectively.

- For the output layer (regression model):

$$\mathbf{d}^3 = \begin{bmatrix} d_1^3 \\ d_2^3 \end{bmatrix} = \begin{bmatrix} y_1 - t_1 \\ y_2 - t_2 \end{bmatrix}$$

$$\partial \mathbf{W}^3 = \mathbf{d}^3 (\mathbf{Z}^2)^T \qquad (2,4) \leftarrow (2,1) \times (1,4)$$

- For the second hidden layer:

$$\mathbf{d}^2 = \begin{bmatrix} d_0^2 \\ d_1^2 \\ d_2^2 \\ d_3^2 \end{bmatrix} = \text{diag}(\partial \mathbf{A}^2)(\mathbf{W}^3)^T \mathbf{d}^3 \qquad (4,1) \leftarrow (4,4) \times (4,2) \times (2,1)$$

where

$$\partial \mathbf{Z}^2 = (\mathbf{W}^3)^T \mathbf{d}^3$$

$$\text{diag}(\partial \mathbf{A}^2) = \begin{bmatrix} \sigma'(a_0^2) & 0 & 0 & 0 \\ 0 & \sigma'(a_0^2) & 0 & 0 \\ 0 & 0 & \sigma'(a_2^2) & 0) \\ 0 & 0 & 0 & \sigma'(a_3^2) \end{bmatrix}$$

$$\partial \mathbf{W}^2 = \mathbf{d}^2 (\mathbf{Z}^1)^T \qquad (4,5) \leftarrow (4,1) \times (1,5)$$

- For the first hidden layer

$$\mathbf{d}^1 = \begin{bmatrix} d_0^1 \\ d_1^1 \\ d_2^1 \\ d_3^1 \\ d_5^1 \end{bmatrix} = \text{diag}(\partial \mathbf{A}^1)(\mathbf{W}^2)^T \mathbf{d}^2 \qquad (4,1) \leftarrow (5,5) \times (5,4) \times (4,1)$$

where

$$\partial \mathbf{Z}^1 = (\mathbf{W}^2)^T \mathbf{d}^2$$

$$\partial \mathbf{W}^1 = \mathbf{d}^1 \mathbf{x}^T \qquad (5,3) \leftarrow (5,1) \times (1,3)$$

- The gradient descent update using an α learning rate is the following.

$$\mathbf{W}^1 \leftarrow \mathbf{W}^l - \alpha \partial \mathbf{W}^l, \qquad l = 1,2,3.$$

Box 3: Multivariate Gaussian

The probability density function of a multivariate Gaussian vector is

$$p(x) = \frac{|\mathbf{\Lambda}|^{1/2}}{(2\pi)^{d/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \mathbf{\Lambda}(\mathbf{x}-\boldsymbol{\mu})\right),$$

- $\mathbf{x} \in \mathbb{R}^d$: the random vector.
- $\boldsymbol{\mu} \in \mathbb{R}^d$: the mean vector.
- $\mathbf{\Lambda} = \mathbf{\Sigma}^{-1} \in \mathbb{R}^{d\times d}$: the precision matrix (inverse of the covariance matrix $\mathbf{\Sigma}$).
- $|\mathbf{\Lambda}|$: the determinant of the precision matrix.
- $(\mathbf{x}-\boldsymbol{\mu})^\top \mathbf{\Lambda}(\mathbf{x}-\boldsymbol{\mu})$: the quadratic form representing the Mahalanobis distance in terms of precision.

The relationship between variance and precision is given as follows:

- In the **univariate case**, let the variance be σ^2 and the precision be λ. The relationship is:

  $$\beta = \frac{1}{\sigma^2},$$

  or equivalently:

  $$\sigma^2 = \frac{1}{\beta}.$$

- In the **multivariate case**, let the covariance matrix be $\mathbf{\Sigma}$ and the precision matrix be $\mathbf{\Lambda}$. The precision matrix is the inverse of the covariance matrix:

  $$\mathbf{\Lambda} = \mathbf{\Sigma}^{-1}.$$

  Similarly, the covariance matrix can be expressed in terms of the precision matrix:

  $$\mathbf{\Sigma} = \mathbf{\Lambda}^{-1}.$$

In the multivariate case, the variances (diagonal elements of $\mathbf{\Sigma}$) are not directly the reciprocals of the corresponding diagonal elements of $\mathbf{\Lambda}$ because $\mathbf{\Lambda}$ also encodes off-diagonal dependencies (covariances).

Regularization in Neural Networks

The number of input and output units in a neural network is generally determined by the dimensionality of the data set. In contrast, the number M of hidden units is a free parameter that can be adjusted

to give the best predictive performance. Note that M controls the number of parameters (weights and biases) in the network, and so we might expect that in a maximum likelihood setting, there will be an optimum value of M that gives the best generalization performance, corresponding to the optimum balance between under-fitting and over-fitting.

There are, however, other ways to control the complexity of a neural network model to avoid over-fitting. The simplest regularizer is the quadratic, giving a regularized error of the form

$$\begin{aligned} E_\lambda(\mathbf{w}) &= E(\mathbf{w}) + \frac{\lambda}{2}\mathbf{w}^T\mathbf{w} = E(\mathbf{w}) + \frac{\lambda}{2} \mid\mid \mathbf{w} \mid\mid^2 \\ &= \sum_{n=1}^{N} E_n(\mathbf{w}) + \frac{\lambda}{2} \mid\mid \mathbf{w} \mid\mid^2 \end{aligned}$$

Notice that:

$$\begin{aligned} \frac{\partial E_n^\lambda}{\partial w_{ji}^l} &= \delta_j^l z_i^{l-1} + \lambda \frac{\partial}{\partial w_{ji}^l} \mid\mid \mathbf{w} \mid\mid^2 . \\ &= \delta_j^l z_i^{l-1} + \lambda w_{ji}^l, \end{aligned}$$

or in matrix notation

$$\partial \mathbf{W}^l = \mathbf{d}^l (\mathbf{Z}^{l-1})^T + \lambda \mathbf{W}^l,$$

Recall that

$$\delta_l^j = h'(a_j^l) \sum_{k=1}^{M_{l+1}} \delta_k^{l+1} w_{kj}^{l+1}.$$

This particular choice of regularizer is known in the machine learning literature as weight decay because, in sequential learning algorithms, it encourages weight values to decay toward zero unless supported by the data. In statistics, it provides an example of a parameter shrinkage method because it shrinks parameter values. It has the advantage that the error function remains a quadratic function of $\mathbf{w}$.

Regularization Types

$$\text{Loss}_{L1} = \text{Loss} + \lambda \sum_{i=1}^{n} |w_i|$$

$$\text{Loss}_{L2} = \text{Loss} + \frac{\lambda}{2} \sum_{i=1}^{n} w_i^2$$

$$\text{Loss}_{EN} = \text{Loss} + \lambda \left(\alpha \sum_{i=1}^{n} |w_i| + \frac{1-\alpha}{2} \sum_{i=1}^{n} w_i^2 \right)$$

Gradient Descent

This document provides a comprehensive overview of gradient descent methods used in training neural networks, including batch gradient descent, stochastic gradient descent, mini-batch gradient descent, and block-wise optimization techniques.

Batch Gradient Descent

Mathematical Formulation

Batch gradient descent computes the gradient using the entire training dataset for each parameter update.

Loss Function:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i; \theta), y_i) \tag{3}$$

where:

- θ represents the model parameters
- N is the total number of training examples
- ℓ is the loss function for a single example
- $f(x; \theta)$ is the neural network output

Gradient Computation:

$$\nabla L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla \ell(f(x_i; \theta), y_i) \tag{4}$$

Parameter Update Rule:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t) \tag{5}$$

where η is the learning rate.

Python Pseudocode

```python
def batch_gradient_descent(X, y, theta, learning_rate, epochs):
    """
    Batch Gradient Descent

    Args:
        X: input data (N x D)
        y: labels (N x 1)
        theta: parameters
        learning_rate: step size
        epochs: number of iterations
    """
```

```python
    N = len(X)

    for epoch in range(epochs):
        # Compute predictions for ALL data
        predictions = forward_pass(X, theta)

        # Compute loss over entire dataset
        loss = compute_loss(predictions, y)

        # Compute gradient using ALL examples
        gradient = (1/N) * compute_gradient(X, y, theta)

        # Update parameters
        theta = theta - learning_rate * gradient

        if epoch % 100 == 0:
            print(f"Epoch {epoch}, Loss: {loss}")

    return theta
```

Characteristics

- Uses entire dataset for each update
- Stable convergence
- Guaranteed to converge to global minimum for convex functions
- Computationally expensive for large datasets
- High memory requirements

Stochastic Gradient Descent (SGD)

Mathematical Formulation

SGD updates parameters using one randomly selected training example at a time.

Single Sample Loss:

$$\ell_i(\theta) = \ell(f(x_i;\theta), y_i) \tag{6}$$

Stochastic Gradient: The gradient is computed for a single randomly selected sample i_t at iteration t:

$$\nabla \ell_{i_t}(\theta) = \nabla \ell(f(x_{i_t};\theta), y_{i_t}) \tag{7}$$

Parameter Update Rule:

$$\theta_{t+1} = \theta_t - \eta \nabla \ell_{i_t}(\theta_t) \tag{8}$$

Expectation Property: The stochastic gradient is an unbiased estimator of the true gradient:

$$\mathbb{E}[\nabla \ell_i(\theta)] = \nabla L(\theta) \tag{9}$$

Python Pseudocode

```python
def stochastic_gradient_descent(X, y, theta, learning_rate, epochs):
    """
    Stochastic Gradient Descent

    Updates parameters after EACH single example
    """
    N = len(X)

    for epoch in range(epochs):
        # Shuffle data each epoch
        indices = np.random.permutation(N)

        epoch_loss = 0
        for i in indices:
            # Get single example
            x_i = X[i]
            y_i = y[i]

            # Forward pass for ONE example
            prediction = forward_pass(x_i, theta)

            # Compute gradient from ONE example
            gradient = compute_gradient_single(x_i, y_i, theta)

            # Update immediately
            theta = theta - learning_rate * gradient

            epoch_loss += compute_loss_single(prediction, y_i)

        if epoch % 100 == 0:
            print(f"Epoch {epoch}, Avg Loss: {epoch_loss/N}")

    return theta
```

Characteristics

- Updates after each single example
- Much faster iterations than batch GD
- Noisy updates help escape local minima
- Low memory requirements
- Requires learning rate scheduling for convergence

- Updates are N times per epoch

Mini-Batch Gradient Descent

Mathematical Formulation

Mini-batch gradient descent is the most commonly used method in practice, combining advantages of both batch and stochastic approaches.

Mini-Batch Loss: For a mini-batch $\mathcal{B}$ of size $|\mathcal{B}|$:

$$L_{\mathcal{B}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell(f(x_i; \theta), y_i) \tag{10}$$

Mini-Batch Gradient:

$$\nabla L_{\mathcal{B}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla \ell(f(x_i; \theta), y_i) \tag{11}$$

Parameter Update Rule:

$$\theta_{t+1} = \theta_t - \eta \nabla L_{\mathcal{B}_t}(\theta_t) \tag{12}$$

where $\mathcal{B}_t$ is the mini-batch at iteration t.

Variance Reduction: The variance of the gradient estimator decreases with batch size:

$$\mathrm{Var}[\nabla L_{\mathcal{B}}(\theta)] = \frac{\mathrm{Var}[\nabla \ell_i(\theta)]}{|\mathcal{B}|} \tag{13}$$

Common batch sizes: $|\mathcal{B}| \in \{32, 64, 128, 256\}$

Python Pseudocode

```
def minibatch_gradient_descent(X, y, theta, learning_rate,
                               epochs, batch_size=32):
    """
    Mini-Batch Gradient Descent (Most Common in Practice)

    Args:
        batch_size: number of examples per update
    """
    N = len(X)

    for epoch in range(epochs):
        # Shuffle data
        indices = np.random.permutation(N)

        # Divide into mini-batches
```

```python
        for start_idx in range(0, N, batch_size):
            end_idx = min(start_idx + batch_size, N)
            batch_indices = indices[start_idx:end_idx]

            # Get mini-batch
            X_batch = X[batch_indices]
            y_batch = y[batch_indices]

            # Forward pass on batch
            predictions = forward_pass(X_batch, theta)

            # Compute gradient over batch
            gradient = (1/len(batch_indices)) * \
                       compute_gradient(X_batch, y_batch, theta)

            # Update parameters
            theta = theta - learning_rate * gradient

        # Evaluate on full dataset periodically
        if epoch % 10 == 0:
            full_loss = evaluate(X, y, theta)
            print(f"Epoch {epoch}, Loss: {full_loss}")

    return theta
```

Characteristics

- Best balance between stability and computational efficiency
- More stable than SGD, faster than batch GD
- Efficient GPU utilization through parallelization
- Standard choice for deep learning applications
- Number of updates per epoch: $\lceil N/|\mathcal{B}| \rceil$

Block-Wise Optimization

Mathematical Formulation

Block-wise optimization updates subsets of parameters separately, which is useful for very large networks or distributed training.

Parameter Partitioning: Divide parameters into K blocks:

$$\theta = [\theta_1, \theta_2, \ldots, \theta_K] \tag{14}$$

Block Coordinate Descent: At iteration t, select block j and update:

$$\theta_j^{(t+1)} = \theta_j^{(t)} - \eta \nabla_{\theta_j} L(\theta_1^{(t+1)}, \ldots, \theta_{j-1}^{(t+1)}, \theta_j^{(t)}, \theta_{j+1}^{(t)}, \ldots, \theta_K^{(t)}) \tag{15}$$

All other blocks remain fixed:

$$\theta_i^{(t+1)} = \theta_i^{(t)} \quad \forall i \neq j \tag{16}$$

Layer-Wise Updates: For neural networks, blocks typically correspond to layers. For layer l with weights $W^{(l)}$ and biases $b^{(l)}$:

$$W_{t+1}^{(l)} = W_t^{(l)} - \eta \nabla_{W^{(l)}} L(\theta) \tag{17}$$

$$b_{t+1}^{(l)} = b_t^{(l)} - \eta \nabla_{b^{(l)}} L(\theta) \tag{18}$$

Alternating Optimization: Cycle through blocks in a fixed or random order:

$$1 \to 2 \to \cdots \to K \to 1 \to \cdots \tag{19}$$

Python Pseudocode

```python
def block_coordinate_descent(X, y, network_params, learning_rate,
                             epochs, batch_size=32):
    """
    Block-Wise Optimization for Neural Networks

    Updates parameters one layer (block) at a time
    """
    layers = network_params  # List of layer parameters
    N = len(X)

    for epoch in range(epochs):
        indices = np.random.permutation(N)

        for start_idx in range(0, N, batch_size):
            batch_indices = indices[start_idx:start_idx+batch_size]
            X_batch = X[batch_indices]
            y_batch = y[batch_indices]

            # Cycle through layers (blocks)
            for layer_idx, layer_params in enumerate(layers):
                # Forward pass through entire network
                activations = forward_pass_all_layers(X_batch,
                    layers)

                # Compute loss
                loss = compute_loss(activations[-1], y_batch)

                # Backprop to get gradient for THIS layer only
                layer_gradient = compute_layer_gradient(
                    X_batch, y_batch, layers, layer_idx,
                        activations
                )

```

```
                # Update ONLY this layer's parameters
                layers[layer_idx] = update_layer(
                    layer_params, layer_gradient, learning_rate
                )

                # Keep other layers fixed

        if epoch % 10 == 0:
            full_loss = evaluate_network(X, y, layers)
            print(f"Epoch {epoch}, Loss: {full_loss}")

    return layers
```

Parallel Block Updates

```
def block_parallel_sgd(X, y, network_params, learning_rate,
                       epochs, batch_size=32, num_blocks=4):
    """
    Parallel Block-Wise Updates

    Partition network into blocks and update in parallel
    """
    # Partition layers into blocks
    blocks = partition_layers(network_params, num_blocks)

    for epoch in range(epochs):
        indices = np.random.permutation(len(X))

        for start_idx in range(0, len(X), batch_size):
            batch_indices = indices[start_idx:start_idx+batch_size]
            X_batch = X[batch_indices]
            y_batch = y[batch_indices]

            # Compute gradients for all blocks
            gradients = []
            for block in blocks:
                grad = compute_block_gradient(
                    X_batch, y_batch, network_params, block
                )
                gradients.append(grad)

            # Update blocks in parallel (can use multiprocessing)
            for block_idx, (block, grad) in enumerate(
                    zip(blocks, gradients)):
                blocks[block_idx] = update_block(
                    block, grad, learning_rate
                )

            # Reassemble network
            network_params = reassemble_network(blocks)
```

```

    return network_params
```

Benefits

- Reduced memory requirements per update
- Enables parallel computation
- Useful for distributed training
- Applicable to very large networks
- Can handle non-differentiable constraints per block

Comparison of Methods

Method	**Update Freq.**	**Convergence**	**Memory**	**Speed**
Batch GD	Once/epoch	Smooth, stable	High	Slow
SGD	N times/epoch	Noisy	Low	Fast iter.
Mini-Batch	$\lceil N/B \rceil$/epoch	Balanced	Medium	Optimal
Block-Wise	Per block	Variable	Very Low	Variable

Table 1: Comparison of gradient descent methods

Best Practices

Learning Rate Scheduling

- Step decay: $\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}$
- Exponential decay: $\eta_t = \eta_0 e^{-\lambda t}$
- Inverse decay: $\eta_t = \eta_0/(1 + \lambda t)$

Adaptive Methods

Consider using adaptive learning rate methods:
- **Adam:** Combines momentum and RMSprop
- **AdaGrad:** Adapts learning rate per parameter
- **RMSprop:** Uses moving average of squared gradients

Practical Recommendations

- Use mini-batch GD as default (batch size 32–256)
- Normalize inputs (zero mean, unit variance)
- Use batch normalization between layers
- Monitor gradient norms to detect vanishing/exploding gradients
- Apply gradient clipping for RNNs

- Consider block-wise optimization for memory-constrained scenarios
- Use data augmentation to prevent overfitting

Mini-batch gradient descent with adaptive learning rates remains the standard approach for training neural networks. Block-wise optimization offers advantages in distributed and memory-constrained settings. The choice of method depends on dataset size, computational resources, and convergence requirements.

The Curse of Dimensionality: Core Problems

The curse of dimensionality refers to various phenomena that arise when analyzing and organizing data in high-dimensional spaces. As the number of features or dimensions grows, the amount of data needed to generalize accurately grows exponentially, and many intuitions from low-dimensional spaces fail dramatically.

Exponential Growth of Sample Complexity

To maintain a fixed density of data points in a d-dimensional space, the number of required samples grows exponentially:

$$N \propto \epsilon^{-d} \tag{20}$$

where ϵ is the desired spacing between points.

Example: Consider uniformly sampling a unit hypercube $[0, 1]^d$:

- In 1D: To have points every 0.1 units requires $N = 10$ samples
- In 2D: Requires $N = 10^2 = 100$ samples
- In 3D: Requires $N = 10^3 = 1000$ samples
- In dD: Requires $N = 10^d$ samples

For $d = 100$, we would need 10^{100} samples, which exceeds the number of atoms in the universe!

Distance Concentration

Most of the space is far from the origin, creating vast empty regions.

For randomly distributed points in high dimensions, the ratio of the distance to the nearest and farthest neighbor approaches 1:

$$\lim_{d \to \infty} \frac{d_{\text{min}}}{d_{\text{max}}} \to 1 \tag{21}$$

Mathematical Analysis: For points uniformly distributed in a d-dimensional unit hypercube, the expected squared Euclidean distance is:

$$\mathbb{E}[\|x - x'\|^2] = \frac{d}{6} \tag{22}$$

The standard deviation scales as:

$$\mathrm{SD}[\|x - x'\|^2] = O(\sqrt{d}) \tag{23}$$

Therefore, the coefficient of variation:

$$\frac{\mathrm{SD}}{\mathbb{E}} = \frac{O(\sqrt{d})}{O(d)} = O(d^{-1/2}) \to 0 \tag{24}$$

This means distances become increasingly concentrated around their mean as d increases.

Volume Concentration

In high dimensions, most of the volume of a hypersphere is concentrated near its surface.

Volume Near Surface

Consider a d-dimensional unit hypersphere. The fraction of volume within distance ϵ of the surface is:

$$\frac{V_d(1) - V_d(1-\epsilon)}{V_d(1)} = 1 - (1-\epsilon)^d \tag{25}$$

For $\epsilon = 0.01$ and $d = 100$:

$$1 - (0.99)^{100} \approx 0.634 \tag{26}$$

Over 63% of the volume is within 1% of the surface!

Convex Sets

A set $C \subseteq \mathbb{R}^n$ is **convex** if for any two points $x, y \in C$, the entire line segment connecting them lies in C. Formally:

$$C \text{ is convex} \iff \forall x, y \in C, \forall \lambda \in [0,1] : \lambda x + (1-\lambda) y \in C$$

The expression $\lambda x + (1-\lambda)y$ represents all points on the line segment from x to y:

- When $\lambda = 0$, we get y
- When $\lambda = 1$, we get x
- When $\lambda = 1/2$, we get the midpoint

Intuition: A set is convex if it has "no dents" - you can connect any two points with a straight line that stays inside the set.

Examples:

- All of $\mathbb{R}^n$
- Half-spaces: $\{x : \langle a, x \rangle \leq b\}$

- Balls (open or closed): $\{x : \|x - x_0\| \leq r\}$
- Intervals in $\mathbb{R}$: $[a, b]$, (a, b), $[a, \infty)$
- Ellipsoids, polyhedra, simplices
- Intersection of convex sets (convexity is preserved under intersection)

Non-examples:

- A disk with a hole removed (doughnut shape)
- The union $(0, 1) \cup (2, 3)$ (disconnected)
- A crescent or star shape (has indentations)
- The boundary of a circle (missing interior points)

Hyperplane Separation Theorem

The **Hyperplane Separation Theorem** is a fundamental result in convex analysis that states when a hyperplane can separate two convex sets. Think of two disjoint convex sets as two non-overlapping "blobs" with no dents or holes. The separation theorem guarantees you can always slide a flat plane between them (possibly just touching their boundaries) that keeps them on opposite sides.

If A and B are two non-empty convex sets in $\mathbb{R}^n$, then under certain conditions, there exists a hyperplane that separates them. A hyperplane is defined by:

$$H = \{x \in \mathbb{R}^n : \langle a, x \rangle = b\}$$

where $a \in \mathbb{R}^n$ is a non-zero normal vector and $b \in \mathbb{R}$ is a scalar.

Types of Separation

Separation

Sets A and B are **separated** by hyperplane H if:

$$\langle a, x \rangle \leq b \quad \text{for all } x \in A \tag{27}$$

$$\langle a, x \rangle \geq b \quad \text{for all } x \in B \tag{28}$$

This means A and B lie on opposite sides of the hyperplane (possibly touching it).

Strict Separation

The sets are **strictly separated** if:

$$\langle a, x \rangle < b \quad \text{for all } x \in A \tag{29}$$

$$\langle a, x \rangle > b \quad \text{for all } x \in B \tag{30}$$

Here, there's a positive gap between each set and the hyperplane.

Strong Separation

There exists $\varepsilon > 0$ such that:

$$\langle a, x \rangle \leq b - \varepsilon \quad \text{for all } x \in A \tag{31}$$

$$\langle a, x \rangle \geq b + \varepsilon \quad \text{for all } x \in B \tag{32}$$

Convex Sets: Basic Separation

If A and B are disjoint convex sets with A closed and B compact, then they can be strictly separated.

Convex Sets: Strong Separation

If A and B are disjoint closed convex sets and at least one is compact, then they can be strongly separated.

Convex Sets: Supporting Hyperplane

If A is a convex set and x_0 is a boundary point of A, then there exists a hyperplane through x_0 that supports A (meaning A lies entirely on one side of the hyperplane).

Closed Set

A set $A \subseteq \mathbb{R}^n$ is **closed** if it contains all its limit points. Equivalently:

- The complement $\mathbb{R}^n \setminus A$ is open
- Every convergent sequence $(x_k) \subset A$ has its limit in A
- A equals its closure: $A = \overline{A}$

Examples:

- Closed interval $[a, b]$ in $\mathbb{R}$ (includes endpoints)
- Closed ball $\{x \in \mathbb{R}^n : \|x - x_0\| \leq r\}$
- Half-spaces $\{x \in \mathbb{R}^n : \langle a, x \rangle \leq b\}$
- Singleton sets $\{x_0\}$

Non-examples:

- Open interval (a, b) (missing endpoints)
- Open ball $\{x \in \mathbb{R}^n : \|x - x_0\| < r\}$

Compact Set

A set $K \subseteq \mathbb{R}^n$ is **compact** if every open cover has a finite subcover. In $\mathbb{R}^n$, by the Heine-Borel theorem, this is equivalent to:

K **is compact** $\iff$ K **is closed and bounded**

A set is **bounded** if it's contained in some ball: $\exists R > 0$ such that $K \subseteq \{x : \|x\| \leq R\}$.

Examples:

- Closed and bounded intervals $[a, b]$ in $\mathbb{R}$
- Closed balls $\{x \in \mathbb{R}^n : \|x - x_0\| \leq r\}$
- Any finite set
- Ellipsoids, closed triangles, closed polygons

Non-examples:

- $\mathbb{R}^n$ itself (closed but not bounded)
- $(0, 1]$ (bounded but not closed)
- $\{x \in \mathbb{R}^n : \|x\| \geq 1\}$ (closed but not bounded)

Key property: Every sequence in a compact set has a convergent subsequence whose limit is in the set.

Applications

This theorem has profound applications in:

- Optimization theory
- Economics and game theory
- Machine learning (especially support vector machines)
- Convex analysis and duality theory

The Kernel Trick

The kernel trick is a fundamental technique in machine learning that allows algorithms to operate in high-dimensional feature spaces without explicitly computing the coordinates in that space. Mercer's theorem provides the theoretical foundation for when a kernel function corresponds to a valid inner product in some feature space.

Motivation: The Limitation of Linear Methods

Many machine learning algorithms operate by finding linear relationships in the input space. Given input data $\mathbf{x} \in \mathbb{R}^d$, linear methods seek functions of the form:

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b \tag{33}$$

However, real-world data often exhibits non-linear patterns that cannot be captured by linear models.

Feature Mapping

A feature map is a function $\phi : \mathcal{X} \to \mathcal{H}$ that maps input data from the original input space $\mathcal{X}$ to a (typically higher-dimensional) feature space $\mathcal{H}$.

For example, consider mapping $\mathbf{x} \in \mathbb{R}^2$ to a higher-dimensional space:

$$\phi(\mathbf{x}) = \phi([x_1, x_2]) = [x_1^2, \sqrt{2}x_1x_2, x_2^2]^T \tag{34}$$

In the feature space, we can find linear relationships:

$$f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) + b \tag{35}$$

which corresponds to a non-linear function in the original space.

The Problem: Computational Complexity

Challenge: For many useful feature maps, the dimension of $\mathcal{H}$ can be very large or even infinite. Computing $\phi(\mathbf{x})$ explicitly becomes:

- Computationally expensive
- Memory intensive
- Sometimes impossible (infinite dimensions)

The Kernel Trick Solution

A kernel function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a function that computes the inner product of two points in feature space without explicitly computing the feature map:

$$k(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle_{\mathcal{H}} \tag{36}$$

Key Insight: Many algorithms only need inner products between feature vectors, not the feature vectors themselves. The kernel trick allows us to compute these inner products efficiently.

Example: Polynomial Kernel

Consider the 2-dimensional input space $\mathbf{x} = [x_1, x_2]^T$.

Explicit Feature Map:

$$\phi(\mathbf{x}) = [x_1^2, \sqrt{2}x_1x_2, x_2^2]^T \tag{37}$$

Computing Inner Product Explicitly:

$$\begin{aligned} \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle &= x_1^2 {x_1'}^2 + 2x_1x_2x_1'x_2' + x_2^2 {x_2'}^2 && (38) \\ &= (x_1x_1' + x_2x_2')^2 && (39) \\ &= (\mathbf{x}^T\mathbf{x}')^2 && (40) \end{aligned}$$

Using the Kernel:

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T\mathbf{x}')^2 \tag{41}$$

Computational Advantage:

- Explicit computation: 3 multiplications + 2 additions (in feature space)
- Kernel computation: 2 multiplications + 1 addition + 1 squaring (in input space)

For higher dimensions, the savings become significant.

General Polynomial Kernel

The general polynomial kernel of degree d is:

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + c)^d \tag{42}$$

where $c \geq 0$ is a constant. This implicitly computes inner products in a feature space of dimension $\binom{n+d}{d}$, where n is the input dimension.

Common Kernel Functions

1. **Linear Kernel:**

$$k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}' \tag{43}$$

2. **Polynomial Kernel:**

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + c)^d \tag{44}$$

3. **Gaussian (RBF) Kernel:**

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right) \tag{45}$$

   This corresponds to an *infinite-dimensional* feature space.

4. **Sigmoid Kernel:**

$$k(\mathbf{x}, \mathbf{x}') = \tanh(\alpha \mathbf{x}^T \mathbf{x}' + c) \tag{46}$$

5. **Laplacian Kernel:**

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|}{\sigma}\right) \tag{47}$$

Mercer's Theorem

Motivation

Not every function $k(\mathbf{x}, \mathbf{x}')$ corresponds to a valid inner product in some feature space. Mercer's theorem provides the conditions under which a kernel function is valid.

Preliminaries

A kernel $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is positive semi-definite (or positive definite) if for any finite set of points $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\} \subset \mathcal{X}$ and any $\mathbf{c} = [c_1, c_2, \ldots, c_n]^T \in \mathbb{R}^n$:

$$\sum_{i=1}^{n} \sum_{j=1}^{n} c_i c_j k(\mathbf{x}_i, \mathbf{x}_j) \geq 0 \tag{48}$$

Equivalently, the **Gram matrix** (or kernel matrix):

$$K = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \cdots & k(\mathbf{x}_1, \mathbf{x}_n) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \cdots & k(\mathbf{x}_2, \mathbf{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & k(\mathbf{x}_n, \mathbf{x}_2) & \cdots & k(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix} \tag{49}$$

must be positive semi-definite, i.e., $\mathbf{c}^T K \mathbf{c} \geq 0$ for all $\mathbf{c} \in \mathbb{R}^n$.

Mercer's Theorem

Let $\mathcal{X}$ be a compact subset of $\mathbb{R}^d$ and $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a continuous, symmetric function. Then k is a positive semi-definite kernel if and only if it can be expressed as:

$$k(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^{\infty} \lambda_i \phi_i(\mathbf{x}) \phi_i(\mathbf{x}') \tag{50}$$

where $\lambda_i \geq 0$ are eigenvalues and ϕ_i are the corresponding eigenfunctions of the integral operator:

$$T_k[f](\mathbf{x}) = \int_{\mathcal{X}} k(\mathbf{x}, \mathbf{x}') f(\mathbf{x}') d\mathbf{x}' \tag{51}$$

with $\sum_{i=1}^{\infty} \lambda_i < \infty$.

Interpretation

Mercer's theorem tells us:

1. A function k is a valid kernel if and only if it is positive semi-definite
2. Every valid kernel can be expressed as an inner product in some (possibly infinite-dimensional) feature space
3. The feature map can be constructed as:

$$\phi(\mathbf{x}) = [\sqrt{\lambda_1}\phi_1(\mathbf{x}), \sqrt{\lambda_2}\phi_2(\mathbf{x}), \sqrt{\lambda_3}\phi_3(\mathbf{x}), \ldots]^T \tag{52}$$

4. Then: $k(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$

Practical Implications

Checking if a Kernel is Valid: To verify if k is a valid kernel, we need to check if it is:

- Symmetric: $k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}', \mathbf{x})$
- Positive semi-definite: The Gram matrix is PSD for any finite set of points

Constructing New Kernels: Mercer's theorem guarantees that the following operations preserve positive semi-definiteness:

If k_1 and k_2 are valid kernels, then the following are also valid kernels:

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}') \quad \text{(Sum)} \tag{53}$$

$$k(\mathbf{x}, \mathbf{x}') = \alpha k_1(\mathbf{x}, \mathbf{x}') \quad \text{(Scaling, } \alpha > 0\text{)} \tag{54}$$

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') \cdot k_2(\mathbf{x}, \mathbf{x}') \quad \text{(Product)} \tag{55}$$

$$k(\mathbf{x}, \mathbf{x}') = f(\mathbf{x}) f(\mathbf{x}') \quad \text{(Function product)} \tag{56}$$

$$k(\mathbf{x}, \mathbf{x}') = k_3(k_1(\mathbf{x}, \mathbf{x}')) \quad \text{(Composition, } k_3 \text{ valid)} \tag{57}$$

Example: RBF Kernel via Mercer's Theorem

The Gaussian RBF kernel:

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right) \tag{58}$$

can be expanded as:

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - 2\mathbf{x}^T\mathbf{x}'}{2\sigma^2}\right) \tag{59}$$

$$= \exp\left(-\frac{\|\mathbf{x}\|^2}{2\sigma^2}\right) \exp\left(-\frac{\|\mathbf{x}'\|^2}{2\sigma^2}\right) \exp\left(\frac{\mathbf{x}^T\mathbf{x}'}{\sigma^2}\right) \tag{60}$$

Using the Taylor expansion of the exponential:

$$\exp\left(\frac{\mathbf{x}^T\mathbf{x}'}{\sigma^2}\right) = \sum_{n=0}^{\infty} \frac{1}{n!} \left(\frac{\mathbf{x}^T\mathbf{x}'}{\sigma^2}\right)^n \tag{61}$$

Each term can be written as an inner product in feature space, showing that the RBF kernel corresponds to an infinite-dimensional feature space.

The Kernel Trick Example

Feature Map

The explicit feature map transforms 1D input to 2D:

$$\phi : \mathbb{R} \to \mathbb{R}^2, \quad \phi(x) = \begin{bmatrix} x \\ x^2 \end{bmatrix}$$

Inner Product in Feature Space

The inner product between two mapped points is:

$$\langle \phi(x), \phi(y) \rangle = \begin{bmatrix} x \\ x^2 \end{bmatrix}^T \begin{bmatrix} y \\ y^2 \end{bmatrix} \tag{62}$$

$$= xy + x^2y^2 \tag{63}$$

$$= xy + (xy)^2 \tag{64}$$

Applications in Machine Learning

Support Vector Machines (SVMs)

The dual formulation of SVM only requires kernel evaluations:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j) \tag{65}$$

subject to constraints on α_i.

The decision function becomes:

$$f(\mathbf{x}) = \text{sign}\left(\sum_{i=1}^{n} \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b \right) \tag{66}$$

Kernel Ridge Regression

The solution to ridge regression in feature space:

$$\hat{\mathbf{w}} = \left(\Phi^T \Phi + \lambda I \right)^{-1} \Phi^T \mathbf{y} \tag{67}$$

can be expressed using kernels:

$$f(\mathbf{x}) = \mathbf{k}^T (K + \lambda I)^{-1} \mathbf{y} \tag{68}$$

where $\mathbf{k} = [k(\mathbf{x}_1, \mathbf{x}), \ldots, k(\mathbf{x}_n, \mathbf{x})]^T$ and $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.

Kernel PCA

Principal Component Analysis in feature space can be performed using only kernel evaluations, allowing non-linear dimensionality reduction.

Summary

The Kernel Trick

- Allows algorithms to operate in high-dimensional (possibly infinite) feature spaces

- Only requires computing inner products via kernel functions
- Avoids explicit feature map computation
- Makes non-linear methods computationally feasible

Mercer's Theorem

- Provides theoretical foundation for valid kernels
- A kernel is valid if and only if it is positive semi-definite
- Guarantees the existence of a feature space and a feature map
- Enables construction of new kernels from existing ones

Key Relationship

$$k(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle_{\mathcal{H}} \quad \Leftrightarrow \quad k \text{ is positive semi-definite} \tag{69}$$

This elegant mathematical framework has enabled the development of powerful non-linear machine learning algorithms that remain computationally tractable.

Support Vector Machines

Support Vector Machines (SVMs) are powerful supervised learning algorithms used for classification and regression. The fundamental idea behind SVM classification is to find an optimal hyperplane that separates data points of different classes with the maximum margin.

The Binary Classification Problem

Given a training dataset $\{(x_i, y_i)\}_{i=1}^n$ where:
- $x_i \in \mathbb{R}^d$ are the feature vectors
- $y_i \in \{-1, +1\}$ are the class labels

The goal is to find a decision function that correctly classifies new, unseen data points.

Linear SVM: The Separating Hyperplane

Decision Function

A hyperplane in $\mathbb{R}^d$ is defined by:

$$f(x) = \langle w, x \rangle + b = 0$$

where $w \in \mathbb{R}^d$ is the normal vector to the hyperplane and $b \in \mathbb{R}$ is the bias term.

The classification rule is:

$$\hat{y} = \text{sign}(f(x)) = \text{sign}(\langle w, x \rangle + b) = \begin{cases} +1 & \text{if } \langle w, x \rangle + b > 0 \\ -1 & \text{if } \langle w, x \rangle + b < 0 \end{cases}$$

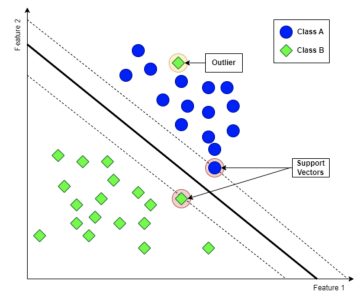


Figure 3: Separable data.

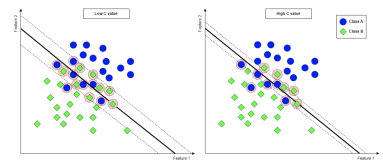


Figure 4: Non-separable data.

Maximum Margin Principle

Among all hyperplanes that separate the two classes, SVM seeks the one with the **maximum margin**. The margin is the distance between the hyperplane and the nearest data point from either class.

For a linearly separable dataset, we want:

$$y_i(\langle w, x_i \rangle + b) \geq 1 \quad \text{for all } i = 1, \ldots, n$$

The margin width is $\frac{2}{\|w\|}$. Therefore, maximizing the margin is equivalent to minimizing $\|w\|^2$.

Optimization Problem (Hard Margin)

The SVM optimization problem for linearly separable data is:

$$\min_{w,b} \quad \frac{1}{2}\|w\|^2 \tag{70}$$

$$\text{subject to} \quad y_i(\langle w, x_i \rangle + b) \geq 1, \quad i = 1, \ldots, n \tag{71}$$

This is a convex quadratic programming problem.

Support Vectors

The **support vectors** are the training points that lie exactly on the margin boundaries, i.e., those for which:

$$y_i(\langle w, x_i \rangle + b) = 1$$

These are the critical points that define the optimal hyperplane. Removing any other points would not change the solution.

Soft Margin SVM

Real-world data is rarely perfectly separable. The **soft margin SVM** introduces slack variables $\xi_i \geq 0$ to allow some misclassifications:

$$\min_{w,b,\xi} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i \tag{72}$$

$$\text{subject to} \quad y_i(\langle w, x_i \rangle + b) \geq 1 - \xi_i \tag{73}$$

$$\xi_i \geq 0, \quad i = 1, \ldots, n \tag{74}$$

The parameter $C > 0$ controls the trade-off between:

- Maximizing the margin (small $\|w\|$)
- Minimizing classification errors (small $\sum \xi_i$)
- **Large C**: Smaller margin, fewer misclassifications (risk of overfitting)
- **Small C**: Larger margin, more misclassifications (more regularization)

Kernel Trick for Non-Linear Classification

When data is not linearly separable in the original space, SVMs use the **kernel trick** to implicitly map data to a higher-dimensional feature space where it becomes separable.

Kernel Function

A kernel function $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ computes the inner product in a transformed feature space:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle$$

where $\phi : \mathbb{R}^d \to \mathcal{H}$ maps to a higher-dimensional (possibly infinite) feature space $\mathcal{H}$.

The decision function becomes:

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i k(x_i, x) + b$$

where α_i are the Lagrange multipliers (non-zero only for support vectors).

Connection to Hyperplane Separation Theorem

SVMs are a direct application of the hyperplane separation theorem:

- The two classes form convex hulls in the feature space
- The separation theorem guarantees a separating hyperplane exists (if classes don't overlap)
- SVM finds the *optimal* such hyperplane by maximizing the margin
- This maximizes generalization to unseen data

Key Properties

- **Convex optimization:** Guaranteed global optimum
- **Sparse solution:** Only support vectors matter
- **Kernel flexibility:** Handles non-linear boundaries
- **Regularization:** C parameter controls overfitting
- **High-dimensional data:** Effective even when $d > n$

Advantages and Limitations

Advantages:
- Effective in high-dimensional spaces
- Memory efficient (uses only support vectors)
- Versatile through different kernel functions
- Strong theoretical foundations

Limitations:
- Computationally intensive for large datasets ($O(n^2)$ to $O(n^3)$)
- Sensitive to kernel and hyperparameter selection
- No probabilistic interpretation (without extensions)
- Primarily designed for binary classification

Appendix

The XOR Problem

Consider 2D input data with the XOR pattern:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2$$

This is not linearly separable in the original space.

RBF Feature Map (Explicit Approximation)

Using landmark points $\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_m$, the RBF feature map is:

$$\phi(\mathbf{x}) = \begin{bmatrix} \exp\left(-\gamma\|\mathbf{x} - \mathbf{c}_1\|^2\right) \\ \exp\left(-\gamma\|\mathbf{x} - \mathbf{c}_2\|^2\right) \\ \vdots \\ \exp\left(-\gamma\|\mathbf{x} - \mathbf{c}_m\|^2\right) \end{bmatrix} \in \mathbb{R}^m$$

where $\gamma > 0$ is the bandwidth parameter.

RBF Kernel (Infinite Dimensional)

The true RBF kernel corresponds to an **infinite-dimensional** feature space:

$$K(\mathbf{x}, \mathbf{y}) = \exp\left(-\gamma\|\mathbf{x} - \mathbf{y}\|^2\right)$$

Expanding the squared distance:

$$K(\mathbf{x}, \mathbf{y}) = \exp\left(-\gamma(\|\mathbf{x}\|^2 - 2\mathbf{x}^T\mathbf{y} + \|\mathbf{y}\|^2)\right) \tag{75}$$

$$= \exp(-\gamma\|\mathbf{x}\|^2) \cdot \exp(2\gamma\mathbf{x}^T\mathbf{y}) \cdot \exp(-\gamma\|\mathbf{y}\|^2) \tag{76}$$

Using Taylor expansion of $\exp(2\gamma \mathbf{x}^T \mathbf{y})$:

$$K(\mathbf{x}, \mathbf{y}) = \sum_{n=0}^{\infty} \frac{(2\gamma)^n}{n!} (\mathbf{x}^T \mathbf{y})^n \cdot \exp(-\gamma(\|\mathbf{x}\|^2 + \|\mathbf{y}\|^2))$$

This represents an inner product in an **infinite-dimensional** Hilbert space!

The Kernel Trick for RBF

- **Explicit feature map:** Would require infinite dimensions! Impossible to compute.
- **Kernel function:** $K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|^2)$ [Just one exponential!]

Key insight: The kernel trick allows us to work in infinite-dimensional feature spaces by only computing pairwise similarities!

Properties of RBF Kernel

1. $K(\mathbf{x}, \mathbf{x}) = 1$ (self-similarity is always 1)
2. $0 < K(\mathbf{x}, \mathbf{y}) \leq 1$ (similarity decreases with distance)
3. As $\|\mathbf{x} - \mathbf{y}\| \to \infty$, $K(\mathbf{x}, \mathbf{y}) \to 0$
4. γ controls the "width" of the kernel:
   - Large γ: narrow kernel, local influence
   - Small γ: wide kernel, global influence

Concentric Circles Example

Problem Setup

Consider data distributed in two concentric circles in $\mathbb{R}^2$:

- **Class 1 (Inner):** $\|x\| \approx r_1 = 1.0$
- **Class 2 (Outer):** $\|x\| \approx r_2 = 2.5$

Input: $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2$

No linear classifier (straight line) can separate these circles!

Radial Feature Map

A simple yet effective feature map that exploits the radial structure:

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1 \\ x_2 \\ \|x\|^2 \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{bmatrix} \in \mathbb{R}^3$$

Key insight: The third coordinate captures the distance from origin!

Linear Separability in Feature Space

In the feature space $\mathbb{R}^3$, we can use a hyperplane:

$$h(\phi(\mathbf{x})) = w_1 x_1 + w_2 x_2 + w_3(x_1^2 + x_2^2) + b = 0$$

For concentric circles, we can choose $w_1 = w_2 = 0$ and $w_3 = 1$:

$$h(\phi(\mathbf{x})) = x_1^2 + x_2^2 + b = \|x\|^2 + b$$

Setting $b = -\left(\frac{r_1^2 + r_2^2}{2}\right)$, the decision boundary becomes:

$$\|x\|^2 = \frac{r_1^2 + r_2^2}{2}$$

This is a **circle** in the original space but a **hyperplane** in the feature space!

Corresponding Kernel

The inner product in feature space is:

$$\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle = \begin{bmatrix} x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{bmatrix}^T \begin{bmatrix} y_1 \\ y_2 \\ y_1^2 + y_2^2 \end{bmatrix} \tag{77}$$

$$= x_1 y_1 + x_2 y_2 + (x_1^2 + x_2^2)(y_1^2 + y_2^2) \tag{78}$$

$$= \mathbf{x} \cdot \mathbf{y} + \|\mathbf{x}\|^2 \|\mathbf{y}\|^2 \tag{79}$$

RBF Kernel for Concentric Circles

The RBF kernel also works well:

$$K(\mathbf{x}, \mathbf{y}) = \exp\left(-\gamma \|\mathbf{x} - \mathbf{y}\|^2\right)$$

For concentric circles:

- Points on the same circle have small $\|\mathbf{x} - \mathbf{y}\| \Rightarrow$ high similarity
- Points on different circles have large $\|\mathbf{x} - \mathbf{y}\| \Rightarrow$ low similarity

Geometric Interpretation

1. **Original space:** Circular decision boundary (nonlinear)
2. **Feature space:** Planar decision boundary (linear)
3. **Transform:** Circles $\rightarrow$ separated regions along the $\|x\|^2$ axis

The kernel trick allows us to work with circular boundaries while only computing linear separators in a higher-dimensional space!

Why This Works

The key property:

$$r_1 < r < r_2 \iff r_1^2 < r^2 < r_2^2$$

By adding $\|x\|^2$ as a feature, we effectively "lift" points based on their distance from the origin, making them linearly separable in 3D!

Appendix

Here is the LaTeX code that derives the forward and backward propagation equations for your specific MLP architecture, including bias terms, using both coordinate and matrix representations.

Notation and Architecture

We consider a Multi-Layer Perceptron (MLP) with:

- Input layer: 2 neurons (x_1, x_2) plus a bias unit $(x_0 = 1)$.
- First hidden layer: 4 neurons $(z_1^{(1)} \dots z_4^{(1)})$ plus a bias unit $(z_0^{(1)} = 1)$. The weighted sum before activation is $h_j^{(1)}$.
- Second hidden layer: 3 neurons $(z_1^{(2)} \dots z_3^{(2)})$ plus a bias unit $(z_0^{(2)} = 1)$. The weighted sum before activation is $h_k^{(2)}$.
- Output layer: 2 neurons $(z_1^{(3)}, z_2^{(3)})$. The weighted sum before activation is $h_l^{(3)}$.

All non-bias neurons use the sigmoid activation function: $z = \sigma(h) = \frac{1}{1+e^{-h}}$.

We denote:

- $W^{(1)}$: weights from input to first hidden layer (size 4×3 because 4 hidden neurons each receive 2 inputs + bias).
- $W^{(2)}$: weights from first hidden to second hidden layer (size 3×5 because 3 hidden neurons each receive 4 inputs + bias).
- $W^{(3)}$: weights from second hidden to output layer (size 2×4 because 2 outputs each receive 3 inputs + bias).

Forward Propagation

Coordinate Form (First Hidden Layer)

For each neuron $j = 1, \dots, 4$ in the first hidden layer:

$$h_j^{(1)} = w_{j0}^{(1)} \cdot 1 + w_{j1}^{(1)} x_1 + w_{j2}^{(1)} x_2 = \sum_{i=0}^{2} w_{ji}^{(1)} x_i$$

$$z_j^{(1)} = \sigma\left(h_j^{(1)}\right) = \frac{1}{1 + e^{-h_j^{(1)}}}$$

where $x_0 = 1$ is the bias input.

Coordinate Form (Second Hidden Layer)

For each neuron $k = 1, \ldots, 3$ in the second hidden layer:

$$h_k^{(2)} = w_{k0}^{(2)} \cdot 1 + \sum_{j=1}^{4} w_{kj}^{(2)} z_j^{(1)} = \sum_{j=0}^{4} w_{kj}^{(2)} z_j^{(1)}$$

$$z_k^{(2)} = \sigma\left(h_k^{(2)}\right)$$

where $z_0^{(1)} = 1$ is the bias activation from first hidden layer.

Coordinate Form (Output Layer)

For each output neuron $l = 1, 2$:

$$h_l^{(3)} = w_{l0}^{(3)} \cdot 1 + \sum_{k=1}^{3} w_{lk}^{(3)} z_k^{(2)} = \sum_{k=0}^{3} w_{lk}^{(3)} z_k^{(2)}$$

$$z_l^{(3)} = \sigma\left(h_l^{(3)}\right)$$

where $z_0^{(2)} = 1$ is the bias activation from second hidden layer.

Matrix Representation

Define the augmented input vectors (including bias):

$$\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix}, \quad \mathbf{z}^{(1)} = \begin{bmatrix} 1 \\ z_1^{(1)} \\ z_2^{(1)} \\ z_3^{(1)} \\ z_4^{(1)} \end{bmatrix}, \quad \mathbf{z}^{(2)} = \begin{bmatrix} 1 \\ z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}, \quad \mathbf{z}^{(3)} = \begin{bmatrix} z_1^{(3)} \\ z_2^{(3)} \end{bmatrix}$$

The weight matrices (including bias columns):

$$W^{(1)} = \begin{bmatrix} w_{10}^{(1)} & w_{11}^{(1)} & w_{12}^{(1)} \\ w_{20}^{(1)} & w_{21}^{(1)} & w_{22}^{(1)} \\ w_{30}^{(1)} & w_{31}^{(1)} & w_{32}^{(1)} \\ w_{40}^{(1)} & w_{41}^{(1)} & w_{42}^{(1)} \end{bmatrix}_{4\times 3}, \quad W^{(2)} = \begin{bmatrix} w_{10}^{(2)} & w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} & w_{14}^{(2)} \\ w_{20}^{(2)} & w_{21}^{(2)} & w_{22}^{(2)} & w_{23}^{(2)} & w_{24}^{(2)} \\ w_{30}^{(2)} & w_{31}^{(2)} & w_{32}^{(2)} & w_{33}^{(2)} & w_{34}^{(2)} \end{bmatrix}_{3\times 5}$$

$$W^{(3)} = \begin{bmatrix} w_{10}^{(3)} & w_{11}^{(3)} & w_{12}^{(3)} & w_{13}^{(3)} \\ w_{20}^{(3)} & w_{21}^{(3)} & w_{22}^{(3)} & w_{23}^{(3)} \end{bmatrix}_{2\times 4}$$

Forward propagation in matrix form:

$$\mathbf{h}^{(1)} = W^{(1)}\mathbf{x}, \quad \mathbf{z}^{(1)} = \sigma(\mathbf{h}^{(1)}) \quad \text{(element-wise, ignoring bias row for now)}$$

But careful: $\mathbf{z}^{(1)}$ in matrix form excludes the bias before the next layer. It's easier to keep bias separate.

Let $\mathbf{a}^{(1)} = [z_1^{(1)}, z_2^{(1)}, z_3^{(1)}, z_4^{(1)}]^T$ (no bias). Then:

$$\mathbf{a}^{(1)} = \sigma\left(W^{(1)}\mathbf{x}\right)$$

Then augment with bias: $\mathbf{z}_{\text{aug}}^{(1)} = [1; \mathbf{a}^{(1)}]$.

Similarly:

$$\mathbf{a}^{(2)} = \sigma\left(W^{(2)}\mathbf{z}_{\text{aug}}^{(1)}\right), \quad \mathbf{z}_{\text{aug}}^{(2)} = [1; \mathbf{a}^{(2)}]$$

$$\mathbf{a}^{(3)} = \sigma\left(W^{(3)}\mathbf{z}_{\text{aug}}^{(2)}\right), \quad \mathbf{z}^{(3)} = \mathbf{a}^{(3)}$$

Backward Propagation (Training)

We use the mean squared error loss for a single training sample $(\mathbf{x}, \mathbf{t})$ where $\mathbf{t} = [t_1, t_2]^T$ are the target outputs.

$$E = \frac{1}{2}\sum_{l=1}^{2}(t_l - z_l^{(3)})^2$$

Output Layer Gradients (Coordinate)

Define the error signal for output neuron l:

$$\delta_l^{(3)} = \frac{\partial E}{\partial h_l^{(3)}} = \frac{\partial E}{\partial z_l^{(3)}} \cdot \frac{\partial z_l^{(3)}}{\partial h_l^{(3)}} = \left(z_l^{(3)} - t_l\right) \cdot \sigma'(h_l^{(3)})$$

Since $\sigma'(h) = \sigma(h)(1 - \sigma(h)) = z(1 - z)$:

$$\delta_l^{(3)} = (z_l^{(3)} - t_l) \cdot z_l^{(3)}(1 - z_l^{(3)})$$

Gradient for weights from second hidden layer to output:

$$\frac{\partial E}{\partial w_{lk}^{(3)}} = \delta_l^{(3)} \cdot z_k^{(2)}, \quad \text{where } z_0^{(2)} = 1$$

Second Hidden Layer Gradients (Coordinate)

For neuron k in the second hidden layer ($k = 1, 2, 3$):

$$\delta_k^{(2)} = \frac{\partial E}{\partial h_k^{(2)}} = \left(\sum_{l=1}^{2} \frac{\partial E}{\partial h_l^{(3)}} \cdot \frac{\partial h_l^{(3)}}{\partial z_k^{(2)}}\right) \cdot \frac{\partial z_k^{(2)}}{\partial h_k^{(2)}} = \left(\sum_{l=1}^{2} \delta_l^{(3)} w_{lk}^{(3)}\right) \cdot \sigma'(h_k^{(2)})$$

$$\delta_k^{(2)} = \left(\sum_{l=1}^{2} \delta_l^{(3)} w_{lk}^{(3)}\right) \cdot z_k^{(2)}(1 - z_k^{(2)})$$

Gradients for weights from first hidden layer to second hidden layer:

$$\frac{\partial E}{\partial w_{kj}^{(2)}} = \delta_k^{(2)} \cdot z_j^{(1)}, \quad \text{where } z_0^{(1)} = 1$$

First Hidden Layer Gradients (Coordinate)

For neuron j in the first hidden layer ($j = 1, 2, 3, 4$):

$$\delta_j^{(1)} = \frac{\partial E}{\partial h_j^{(1)}} = \left(\sum_{k=1}^{3} \frac{\partial E}{\partial h_k^{(2)}} \cdot \frac{\partial h_k^{(2)}}{\partial z_j^{(1)}} \right) \cdot \frac{\partial z_j^{(1)}}{\partial h_j^{(1)}} = \left(\sum_{k=1}^{3} \delta_k^{(2)} w_{kj}^{(2)} \right) \cdot \sigma'(h_j^{(1)})$$

$$\delta_j^{(1)} = \left(\sum_{k=1}^{3} \delta_k^{(2)} w_{kj}^{(2)} \right) \cdot z_j^{(1)}(1 - z_j^{(1)})$$

Gradients for weights from input layer to first hidden layer:

$$\frac{\partial E}{\partial w_{ji}^{(1)}} = \delta_j^{(1)} \cdot x_i, \quad \text{where } x_0 = 1$$

Matrix Representation of Backpropagation

Define the error vectors (excluding bias neurons):

$$\boldsymbol{\delta}^{(3)} = \begin{bmatrix} \delta_1^{(3)} \\ \delta_2^{(3)} \end{bmatrix}, \quad \boldsymbol{\delta}^{(2)} = \begin{bmatrix} \delta_1^{(2)} \\ \delta_2^{(2)} \\ \delta_3^{(2)} \end{bmatrix}, \quad \boldsymbol{\delta}^{(1)} = \begin{bmatrix} \delta_1^{(1)} \\ \delta_2^{(1)} \\ \delta_3^{(1)} \\ \delta_4^{(1)} \end{bmatrix}$$

Let $\odot$ denote element-wise multiplication. Then:

$$\boldsymbol{\delta}^{(3)} = (\mathbf{z}^{(3)} - \mathbf{t}) \odot \mathbf{z}^{(3)} \odot (1 - \mathbf{z}^{(3)})$$

$$\boldsymbol{\delta}^{(2)} = \left((W_{\text{no bias}}^{(3)})^T \boldsymbol{\delta}^{(3)} \right) \odot \mathbf{z}^{(2)} \odot (1 - \mathbf{z}^{(2)})$$

where $W_{\text{no bias}}^{(3)}$ is $W^{(3)}$ without the bias column (i.e., columns corresponding to $k = 1, 2, 3$ only), size 2×3.

$$\boldsymbol{\delta}^{(1)} = \left((W_{\text{no bias}}^{(2)})^T \boldsymbol{\delta}^{(2)} \right) \odot \mathbf{z}^{(1)} \odot (1 - \mathbf{z}^{(1)})$$

where $W_{\text{no bias}}^{(2)}$ is $W^{(2)}$ without the bias column (i.e., columns corresponding to $j = 1, 2, 3, 4$ only), size 3×4.

The weight gradient matrices (including bias columns) are:

$$\frac{\partial E}{\partial W^{(3)}} = \boldsymbol{\delta}^{(3)} (\mathbf{z}_{\text{aug}}^{(2)})^T, \quad \frac{\partial E}{\partial W^{(2)}} = \boldsymbol{\delta}^{(2)} (\mathbf{z}_{\text{aug}}^{(1)})^T, \quad \frac{\partial E}{\partial W^{(1)}} = \boldsymbol{\delta}^{(1)} \mathbf{x}^T$$

where $\mathbf{z}_{\text{aug}}^{(2)} = [1, z_1^{(2)}, z_2^{(2)}, z_3^{(2)}]^T$ and $\mathbf{z}_{\text{aug}}^{(1)} = [1, z_1^{(1)}, z_2^{(1)}, z_3^{(1)}, z_4^{(1)}]^T$, and $\mathbf{x} = [1, x_1, x_2]^T$.